ML Techniques

Oversampling/Undersampling

These methods are aimed at trying to actually predict positive cases that are very rary.

- Think about what would happen if we used knn with very rare events.

Don’t just use these techniques if you see imbalanced data! **Many times, they actually won’t be very good**.

- We are essentially biasing the data and hoping that this performs better on future data.

How Imbalanced?

Techniques for imbalanced data: Use them when you have issues!

- Train-Test split does not provide enough observations to evaluate and/or build models
- The models just predict majority class and perform poorly on the minority class

Synthetic Minority Oversampling Technique (SMOTE)

SMOTE tries to add some random variation to avoid too much overfitting.

Note: This is synthetic data! No guarantees this will improve performance … try along with original data to see how performance varies.

Missing Values

ML models generally can’t work with data with missing values. You need to use complete cases.

Note: Many times, we might want to code missing values as their own category. This means they aren’t “missing” anymore!

ML can be used to imputation too though!

Stacking

Stacking involves combining many different models into one overall model.

- Use the predictions from the lower-level models as features.

You can think about this as just weighting the predictions from various models to try to come up with an overall model that works better than any individual model.

Overfitting with Stacking

A bit different from the motivation for Bagging/Boosting

- **Basic idea:** Instead of trying to get an ensemble of similar overfitting models and averaging over them, we want to take the best parts of each different model and combine them

Big difference from other ensemble models we have used: we are combining **different types of models**, rather than the same kind (e.g., random forests uses all trees).

As always, there are concerns about overfitting … but you can address these the same way using CV.

Stacking

To some extent, you can interpret the stacked models using the meta model.

- Example: if it's a linear regression model, **interpret the coefficients** as contribution of lower-level models

However, **this does not provide explanation of overall model**.

- In a way, we lose even more interpretability with this. However, Interpretable ML methods still work.

Our overall goal is **prediction**, so we try to optimize on that.

Stacking

Choosing Models: If a (lower-level) model does not do a good job of predicting the outcome, then it will end up not contributing to the meta model. So, not much downside to choosing lower-level models.

- However, more models contributes to more computation time.

Super Learner

The **Super Learner** will generally perform at least as well as the best model from the meta models.

- Practically speaking, generally does about as well as or slightly better than the best model.